

Using function 'complex_mag_cmp', we can define individual comparison functions for operators '>=', '>', '=', '<', '<=' and the operators themselves, as shown below. For every operator, the commands below define certain properties, which can be exploited by the query optimiser.

```
CREATE FUNCTION complex_mag_ge(complex, complex) RETURNS
BOOL AS $$
begin
    return (complex_mag_cmp($1, $2) => 0);
end;
$$ LANGUAGE PLPGSQL;
CREATE FUNCTION complex_mag_gt(complex, complex) RETURNS
BOOL AS $$
begin
    return (complex_mag_cmp($1, $2) > 0);
end;
$$ LANGUAGE PLPGSQL;
CREATE FUNCTION complex_mag_lt(complex, complex) RETURNS
BOOL AS $$
begin
    return (complex_mag_cmp($1, $2) < 0);
end;
$$ LANGUAGE PLPGSQL;
CREATE FUNCTION complex_mag_le(complex, complex) RETURNS
BOOL AS $$
begin
    return (complex_mag_cmp($1, $2) <= 0);
end;
$$ LANGUAGE PLPGSQL;
CREATE FUNCTION complex_mag_eq(complex, complex) RETURNS
BOOL AS $$
begin
    return (complex_mag_cmp($1, $2) = 0);
end;
$$ LANGUAGE PLPGSQL;

CREATE OPERATOR < (
    leftarg = complex, rightarg = complex, procedure =
complex_mag_lt,
    commutator = > , negator = >= ,
    restrict = scalarlttsel, join = scalarltjoin
);
CREATE OPERATOR <= (
    leftarg = complex, rightarg = complex, procedure =
complex_mag_le,
    commutator = >= , negator = > ,
    restrict = scalarlttsel, join = scalarltjoin
);
CREATE OPERATOR = (
    leftarg = complex, rightarg = complex, procedure =
complex_mag_eq,
    commutator = = ,
    -- leave out negator since we didn't create <> operator
```

```
-- negator = <> ,
    restrict = eqtsel, join = eqjoin
);
CREATE OPERATOR >= (
    leftarg = complex, rightarg = complex, procedure =
complex_mag_ge,
    commutator = <= , negator = < ,
    restrict = scalargtsel, join = scalargtjoin
);
CREATE OPERATOR > (
    leftarg = complex, rightarg = complex, procedure =
complex_mag_gt,
    commutator = < , negator = <= ,
    restrict = scalargtsel, join = scalargtjoin
);
```

You can find the rows in the table 'complex_tab,' which have val > val2 using the '>' operator defined above.

```
SELECT * FROM complex_tab WHERE val > val2;
   val | val2
-----+-----
   (3,4) | (3,3)
   (1 row)
```

Here is another query to find out all the rows (in table 'complex_tab'), which have val2 greater than the sum of the column val from all the rows in the table 'complex_tab'.

```
SELECT * FROM complex_tab WHERE val2 > (SELECT sum(val) FROM
complex_tab);
   val | val2
-----+-----
   (1,1) | (10,2)
   (1 row)
```

One can go further, by defining an operator class, so that indexes can be created and used for faster queries. For simplicity and brevity, we will omit the operator class definitions in this article.

User-defined procedural languages

PostgreSQL extensibility goes beyond other database systems by allowing users to define procedural languages. Procedural languages are not part of SQL syntax, per se, but are languages in which users can write user-defined functions. We have used a built-in procedural language PL/pgSQL provided by the standard distribution of PostgreSQL. By defining their own procedural language, users can put their application logic in a more comfortable language. For example, this feature allows Perl developers to write stored functions and procedures in the database without coming out of their comfortable Perl world. A function written in a user-defined procedural language